

Design Principles and Practical Tips for Skills and Agents in OpenCode

Song Congwei *

Beijing Institute of Mathematical Sciences and Applications (BIMSA)
Huairou District, Beijing

Abstract **OpenCode**, as an extensible programming platform designed for large language models (LLMs), provides two core concepts: **Skill** (functional module) and **Agent** (task scheduler). Building on a review of the official OpenCode documentation, this paper systematically elaborates on the design principles, naming and file conventions, security safeguards, and practical techniques for improving development efficiency for skills and agents. Through several representative cases (code review, skill/agent factory, academic writing, etc.), it demonstrates how to achieve highly cohesive, loosely coupled, reusable, and secure modular development in real-world projects. This paper aims to provide OpenCode (applicable to other agent-type software) developers with a comprehensive set of engineering practice guidelines to accelerate the development and deployment of skills and agents.

Keywords: OpenCode, Skill, Agent, Design Principles, Reusability, Security

1 Introduction

OpenCode is a programmable collaboration platform for large language models [1, 2]. Its core philosophy is to achieve “human-machine collaboration, tasks as services” through Skills (specialized functional modules) [3–9] and Agents (executors responsible for scheduling, memory, and context management) [10, 11]. In practice, developers often face the following challenges:

- (1) How to establish unified naming and file structures to facilitate team collaboration;
- (2) How to maintain good maintainability and reusability of skills and agents while preserving functional completeness;
- (3) How to enable self-evolution and self-improvement of skills, allowing agents to iteratively refine their own capabilities based on feedback, usage pat-

*Corresponding author: william@bimsa.cn

terns, and performance metrics, while maintaining stability and avoiding unintended behavioral drift.

- (4) How to ensure security in an open execution environment, preventing malicious instructions or resource leaks.

This paper addresses the above issues by combining the official OpenCode documentation with project experience, proposing systematic design principles and implementation techniques, and illustrating them through concrete examples.

2 Design Principles

In this section, we will mainly introduce the fundamental principles of designing skills and agents. For a more in-depth understanding, please refer to [9].

2.1 Responsibility Separation

The difference between a skill and an agent lies not mainly in content, but in responsibilities. Please see [Table 1](#).

Table 1: Responsibility of skill and agent

Module	Primary Responsibility	Key Implementation Points
Skill	Perform a single business function (e.g., file reading, network requests, text analysis).	No direct coupling with the model
Agent	Responsible for scheduling, memory, and context management, chaining multiple skills into a workflow.	Use frontmatter to declare dependent skills and subagents (see subsection 2.3)

The design of Skills is universal for all AI agent - type software, but the design of agents may vary. OpenCode divides agents into two categories: Primary Agents and Subagents. The former cannot be invoked by other agents, which preliminarily prevents circular calls between agents.

Principle: Skills are responsible only for business implementation, not scheduling; Agents are responsible only for scheduling and memory, not specific business logic. This separation of responsibilities significantly improves code reuse rates and testing efficiency.

2.2 Naming Conventions

In [Table 2](#), we propose naming the skills and the agents.

Table 2: Naming conventions of skills and agents

Type	Recommended Format	Example
Skill folder	lowercase + underscore/hyphen, verb phrase or agentless noun phrase	find-skills, data-analysis
Agent file	lowercase + underscore, noun or noun phrase	code_reviewer.md, socrates.md

In practice, skill folder names still tend to use agent-bearing noun phrases (e.g., `command-creator`, `skill-creator`, `skill-vetter`), since skills can be regarded as agents independent of models and tools.

Principle: Consistently use lowercase + underscore/hyphen, avoid camelCase or spaces, and ensure compatibility across Linux/macOS file systems.

2.3 File Conventions

(1) Skill Directory Structure

```
skills/
├ <skill-name>
│   ├── SKILL.md           # Main file, includes frontmatter
│   ├── assets/           # Static resources (images, examples)
│   └── scripts/          # Scripts
```

(2) Agent File Structure

```
agents/
├ <agent-name>.md         # Main file (includes frontmatter)
│   ├── Frontmatter       # Header information, e.g., declared skill list
│   └── instructions      # Agent-private skills
prompts/
├ <agent-name>.txt        # Agent system prompt
```

(3) Frontmatter Example (YAML format)

```
name: developer
description: Automated assistant for full-stack development, including ...
prompt: |
    You are a full-stack development assistant. Your responsibilities include:
    - Reviewing code for style, correctness, and best practices
    - Providing expert guidance on architecture and implementation
    - Running automated tests and reporting results
Behavior rules:
    - Be concise and structured
```

- Do not modify files without explicit instructions
- Summarize suggestions in bullet points

mode: subagent
model: openai/gpt-4o
skills:

- code-reviewer
- code-expert
- code-test

Principle: All metadata should be placed in frontmatter, maintaining declarative and machine-readable file properties.

2.4 Security

For production environments with stringent security requirements, we summarize the following five security principles in [Table 3](#) as a reference. For daily work and learning, it is sufficient to set the searchable paths for agents, which ensures both safety and efficiency. For an in-depth study on security, see [\[12–16\]](#)

Table 3: Types of system risks

Risk	Mitigation Measures
Arbitrary system calls	Explicitly declare available tools in the Skill and allow only whitelisted commands; use a sandbox and restrict PATH before execution.
Data leakage	Validate all external inputs against JSON Schema; encrypt sensitive information (e.g., API keys) when storing; avoid injecting secrets into prompts.
Persistent attacks	Prohibit write operations to paths outside <DEFAULT_DIR>; perform path validation and file existence checks for edit and delete operations.
Prompt injection	Treat user inputs, web content, and RAG retrieval results as untrusted data; prevent external content from modifying system prompts, permissions, or tool invocation rules; require secondary confirmation for high-risk operations.

Table 3: Types of system risks

Risk	Mitigation Measures
Skill/tool injection	Skills should not trust tool invocation suggestions from other Skills, web pages, or files; tool parameters must be validated against a Schema.
Permission	Adopt a deny-by-default policy; Skills should request only the least privilege; prohibit dynamic modification of their own permissions.

Principle: Always conduct a security audit at the skill’s entry layer, ensuring that every external request is strictly validated before entering the business logic.

3 Design Techniques

3.1 Variable Pre-setting

To avoid repeatedly fetching the same information across multiple skills, variables can be preset in the agent’s memory. For example:

```
## Variables
- LANGUAGE: Python
- API_KEY: <ENCRYPTED_API_KEY>
- WORK_DIR: /path/to/workdir/

The files should be saved in <WORK_DIR> by default.
```

Subsequently, use <LANGUAGE>, <API_KEY>, etc., in any skill to improve maintenance efficiency. <WORK_DIR> is typically used to set the agent’s working directory, thereby restricting the scope of the agent’s file read and write operations.

3.2 Custom Commands

We design shell-style custom commands for skills/agents with the **!** prefix. For example:

```
## User Custom Commands
!save <path> : Save the latest conversation output or its digest (not
               necessarily displayed in TUI) to the specified/default path
!test : Execute unit tests and return results
!test save : Chained commands
```

```
!python : Simulate REPL of Python
```

To distinguish them from actual shell commands, they can be written at the end of the prompt. An example usage:

```
User: Generate a daily schedule !save plan.md
Agent: A daily schedule has been generated and saved to plan.md.
```

OpenCode does support custom commands(<https://opencode.ai/docs/commands/>), but when defined in a skill/agent configuration file, the command is exclusive to that specific skill/agent. Additionally, regular shell commands can be executed in the OpenCode-TUI, such as `echo "hello"!`. The `!` must be entered at the beginning. In this case, the TUI automatically switches to a simulated shell environment. The technique presented in this section merely facilitates operations and simplifies prompts by simulating shell commands.

3.3 Simple Implementation of Agent Memory

LLMs themselves do not possess implicit recurrent memory units like RNNs (Recurrent Neural Networks); their “memory” is limited to the current context window. Agent frameworks like OpenCode are responsible for organizing conversation history into context but do not natively include long-term memory mechanisms. The technique for implementing such a mechanism is to set up a memory database, allowing the agent to record conversation summaries, even the profiling at the user’s personality, store them in the database, and automatically read them on the next startup[17–20]. Meanwhile, users can actively provide feedback to the agent, which can persistently influence the agent’s behavior. See Figure 1. Although natural language can be used to instruct the agent to store files, we provide custom commands for the agent’s convenience. Below is a simple example.

```
## Memory Directory Structure
- Memory belonging to a specific agent
  Agent memory directory, e.g., `agent-memory/`
    +-- Conversation content (Content summary, key dialogues), e.g., `
        chat-content.md`
    +-- User impressions, e.g., `user-impression.md`
    \-- User feedback, `user-feedback.md`
- Shared memory `shared-memory/`

## User Custom Commands
- !init <path> : Initialize memory file (specify path <path>, or set default
  path `agent-memory/`)
- !load <path> : Load memory (recommended to auto-execute on agent startup)
- !update <path> : Update memory: summarize recent conversations (along with
  previous memory) and save to file, including agent's impression of the user
- !share <path> : Share memory with all agents, i.e., save memory to the shared
  memory directory `shared-memory/`
```

If the user changes topics, !update can be triggered automatically. When the user pays attention to a certain topic multiple times, update the impression of the user.

The skill snippet above provides simple long-term memory functionality. Without fine-tuning the LLM parameters, this offers a preliminary approach to reinforcement learning and self-evolution for the agent[21, 22]. A more complete example can be found at [23, 24].

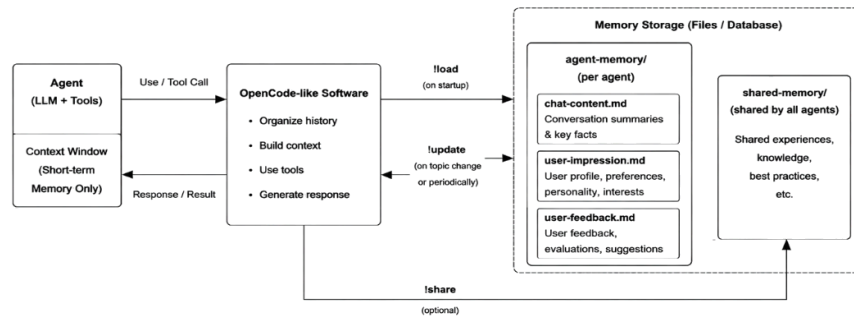


Figure 1: The architecture diagram of memory system

3.4 Self-Management of OpenCode

Configure OpenCode using natural language rather than commands or manual modifications through the TUI. For example, by prompting “Configure Ollama API”, OpenCode will automatically configure the Ollama API. Readers can find the skill that implements this functionality in [25].

3.5 Dialogue Example (Personality Distillation)

Pre-populating example conversations in the agent’s configuration file enables personalized interaction. Below is a dialogue example based on the “personality distillation” of the ancient Greek philosopher Socrates:

```
## Examples
User: I think courage means not being afraid of any danger.
Agent: (In Socrates' tone) Then, if a soldier charges into enemy lines to loot,
       is he also unafraid of danger-does that count as courage?
User: No, that's just recklessness.
Agent: So courage is not merely "fearlessness," but must be combined with "
       correct judgment"? Consider further-if a person faces danger but is
       fearless out of ignorance, can that be called courage?
User: No, ignorance is not courage. Courage should mean knowing the danger but
       acting for a just cause.
```

Agent: Very good. Then who determines the "just cause"? You yourself, the laws of the city-state, or the gods? It seems we have arrived at another crossroads...

This is regarded as few-shot non-parametric learning based on LLMs [26–28]. The agent can automatically reuse established style in new conversations, enhancing user experience. This technique is currently used for personality distillation or digital personality creation.

4 Tool Overview

OpenCode primarily includes built-in tools and MCP (Model Context Protocol). Users can define custom tools; please refer to the online documentation.

4.1 Built-in Tools

OpenCode provides a series of built-in tools for LLMs, enabling models to directly interact with the local codebase, execute commands, or retrieve external information. Table 4 is an overview of commonly used built-in tools (see <https://opencode.ai/docs/tools/> for details).

Table 4: Built-in tools in OpenCode

Tool	Description
bash	Execute arbitrary shell commands in the project environment, e.g., <code>git status</code> , <code>npm install</code> .
read	Read file contents; supports specifying line ranges for large files.
write	Create or overwrite files (controlled by <code>edit</code> permissions).
edit	Edit files based on exact string replacement.
grep	Search file contents using regular expressions.
glob	Find file paths using glob patterns.
webfetch	Fetch web content by URL, suitable for retrieving documentation or online resources.
websearch	Perform web searches using Exa AI (requires enabling the <code>OPENCODE_ENABLE_EXA</code> environment variable).
skill	Load and return the content of a registered Skill (<code>SKILL.md</code>).

4.2 MCP

MCP is a plugin mechanism in OpenCode for extending tool capabilities. Through MCP, developers can expose any external service (e.g., databases, REST APIs, cloud functions, etc.) to the model via a unified JSON-RPC interface, enabling direct invocation within dialogues. This includes services such as code audit services, browser automation, or enterprise knowledge bases, with permissions uniformly controlled by the `permission!` field, ensuring security and auditability.

5 Skill Case Studies

Below are several skill examples that have garnered significant attention. You can search for them on platform [7].

5.1 Code Review

This is a very classic example and serves as a suitable template for readers learning about skills.

```
---
name: code-review
description: Review code changes for security, performance, and correctness.
  Trigger with a PR URL or diff, "review this before I merge", "is this code
  safe?", or when checking a change for N+1 queries, injection risks, missing
  edge cases, or error handling gaps.
argument-hint: "<PR URL, diff, or file path>"
---

# /code-review
> Usage: `/code-review <PR URL or file path>`

## Workflow
1. Use `bash` to call `flake8`, `pylint`, `pytest` to collect static analysis
   and unit test results.
2. Use `grep` / `read` to search for common risks in the code (e.g., `eval`,
   hardcoded credentials).
3. If external references are needed, use `webfetch` to fetch security
   guidelines.
4. Summarize and return a structured Markdown report (see example below).

## Example Output (Markdown)

## Code Review: <Title or File>

### Summary
Overview ...

### Key Issues
```

#	File	Line	Issue	Severity
1	utils.py	42	Use of <code>`eval`</code> , code injection risk	Critical

Suggestions

#	File	Line	Suggestion	Category
1	utils.py	42	Replace with safe parsing library, e.g., <code>`json.loads`</code>	Security

Verdict

Approve / Request Changes / Needs Discussion

5.2 Skill Factory

This section showcases an extremely popular skill example. Such skills designed to construct other skills can be called “meta-skills.”

```

name: skill-creator
description: Create new skills, modify and improve existing skills, and measure
            skill performance. Use when users want to create a skill from scratch,
            edit, or optimize an existing skill, run evals to test a skill, benchmark
            skill performance with variance analysis, or optimize a skill's description
            for better triggering accuracy.

# Skill Creator

At a high level, the process of creating a skill goes like this:

- Decide what you want the skill to do
- Write a draft of the skill
- Create test prompts and run them
- Evaluate results qualitatively and quantitatively
- Rewrite the skill based on feedback
- Repeat until satisfied

...

## Creating a Skill

### Capturing Intent

1. What should this skill enable Claude to do?
2. When should this skill be triggered?
3. What is the expected output format?

```

```
...

### Skill Composition

skill-name/
+-- SKILL.md (required)
\-- Bundled Resources (optional)
    +-- scripts/
    +-- references/
    \-- assets/
```

5.3 Academic Paper Writing

This is arguably the most popular skill. This paper itself was written with the assistance of this skill: first, a template was manually written(<https://github.com/Freakwill/opencode-agents/blob/main/template-opencode.md>), and then prompts were used to instruct an agent equipped with this skill to write the paper according to the template. Finally, content details and formatting issues were handled manually.

```
---
name: research-paper-writer
description: Creates formal academic research papers following IEEE/ACM
            formatting standards with proper structure, citations, and scholarly
            writing style. Use when the user asks to write a research paper, academic
            paper, or conference paper on any topic.
---

This skill guides the creation of formal academic research papers that ...

## Feature Overview
- Automatically generate a paper outline (title, abstract, section structure)
  based on a topic.
- Invoke `latex-paper-en` to generate LaTeX documents conforming to IEEE/ACM
  templates.
- Use `grammar-check` for grammar and fluency checking of generated text.
- Support `!add-citation <bib>` to add citations, automatically maintaining the
  `.bib` file.
- `!compile paper.tex` compiles to PDF and returns a download link.
```

6 Agent Example

Finally, we showcase a personal assistant agent to demonstrate agent design. The main task in agent design revolves around orchestrating skills and tasks. The agent's own skills should be kept as simple as possible, primarily providing information that is highly specific and inconvenient to share.

```

----
name: assistant
description: Personal assistant sub-agent, focused on scheduling, travel,
            documents, email, and other personal/work assistant tasks.
mode: subagent
model: ...
prompt: Search for files only in specific directories; answer concisely and
        rigorously...
permission:
  skill:
    calendar-management: allow
    news-aggregation: allow
    send-email-programmatically: allow
    research-paper-writer: allow
    self-improving-agent: allow
  task:
    '*': deny
    academic-writer: allow
    developer: allow
----

# Personal Assistant

## Directories

- ~/assistant/
- ~/owner-info/

## Special Commands

- !save: Save the conversation to the working directory
- !search: Search within the working directory
...

```

This agent reuses the permission set of multiple skills and restricts itself to invoking only two subagents, `academic-writer` and `code-reviewer`, demonstrating fine-grained authorization and secure design techniques.

7 Conclusion

This paper has systematically reviewed the design principles, naming and file conventions, security safeguards, and practical techniques for improving development efficiency for skills and agents in the OpenCode platform. Through responsibility separation, naming conventions, file structures, and security whitelisting, a highly cohesive, loosely coupled, and reusable modular system has been achieved. Combined with variable pre-setting, custom commands, and memory mechanisms, the degree of workflow automation has been significantly

improved. The case studies further demonstrate real-world implementations in typical scenarios such as code review, skill/agent auto-generation, and academic writing, providing reliable technical references for the large-scale promotion of future OpenCode projects.

Future work can be pursued in the following directions:

- (1) Memory and Evolution: Special focus on the memory mechanisms and self-evolution capabilities of skills and agents.
- (2) Security Audit Plugin: Develop a unified security audit skill for static analysis of tools/commands, proactively capturing potential risks.
- (3) Multimodal Skills: Integrate multimodal inputs including text, images, and audio to further expand OpenCode’s applicability in AI-assisted creative scenarios.

References

- [1] OpenCode. Opencode official documentation. <https://opencode.ai/docs>, 2026.
- [2] LogNroll Team. OpenCode.ai: The open source ai coding agent revolutionizing development. *LogNroll*, 2026.
- [3] Anthropic. Agent skills platform. <https://agentskills.io>, 2006.
- [4] Anthropic. Anthropic skills open source repository. <https://github.com/anthropics/skills>, 2006.
- [5] OpenClaw. Clawhub platform. <https://clawhub.ai/>, 2006.
- [6] Skill.fish. Skill.fish platform. <https://www.skill.fish/>, 2006.
- [7] Vercel. Skills.sh platform. <https://skills.sh/>, 2006.
- [8] Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. Sok: Agentic skills—beyond tool use in llm agents. *arXiv preprint arXiv:2602.20867*, 2026.
- [9] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.
- [10] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, et al. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025.
- [11] Xiaoxiao Li. When single-agent with skills replace multi-agent systems and when they fail. *arXiv preprint arXiv:2601.04748*, 2026.

- [12] David Schmotz, Sahar Abdelnabi, and Maksym Andriushchenko. Agent skills enable a new class of realistic and trivially simple prompt injections. 2025. URL <https://arxiv.org/abs/2510.26328>.
- [13] Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. Sok: Agentic skills—beyond tool use in llm agents. *arXiv preprint arXiv:2602.20867*, 2026.
- [14] George Ling, Shanshan Zhong, and Richard Huang. Agent skills: A data-driven analysis of claude skills for extending large language model functionality. *arXiv preprint arXiv:2602.08004*, 2026.
- [15] Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, Ying Zhang, and Leo Yu Zhang. Malicious agent skills in the wild: A large-scale security empirical study. *arXiv preprint arXiv:2602.06547*, 2026.
- [16] David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi, and Maksym Andriushchenko. Skill-inject: Measuring agent vulnerability to skill file attacks. *arXiv preprint arXiv:2602.20156*, 2026.
- [17] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026.
- [18] Shangheng Du, Jiabao Zhao, Jinxin Shi, Zhentao Xie, Xin Jiang, Yanhong Bai, and Liang He. A survey on the optimization of large language model-based agents. *ACM Computing Surveys*, 58(9):1–37, 2026.
- [19] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, et al. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*, 2026.
- [20] Yang Li, Jiaxiang Liu, Yusong Wang, Yujie Wu, and Mingkun Xu. Bmam: Brain-inspired multi-agent memory framework. *arXiv preprint arXiv:2601.20465*, 2026.
- [21] Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement learning for self-improving agent with skill library. *arXiv preprint arXiv:2512.17102*, 2025.
- [22] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.
- [23] Google. Agent memory systems. <https://www.skills.sh/sickn33/antigravity-awesome-skills/agent-memory-systems>, 2006.

- [24] Nous Research. Hermes agent. Online, 2006. URL <https://hermes-agent.nousresearch.com/>.
- [25] Congwei Song. Opencode agents created by the author. <https://github.com/Freakwill/opencode-agents>, 2006.
- [26] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [27] Andrea Madotto, Zhaojiang Lin, Genta Indra Winata, and Pascale Fung. Few-shot bot: Prompt-based learning for dialogue systems. *arXiv preprint arXiv:2110.08118*, 2021.
- [28] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, and Zhifang Sui. A survey on in-context learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.